Software Engineering - Assignment #04

Smart Campus Room Availability Checker: extension, refactoring and testing

Student	**Talgatbek Abilkhaiyr**
Student ID	24B032048
Course	Software Engineering, Assignment #04
Date	31 October 2026
AI assistant	Claude (Anthropic), model Claude Opus 5.5
Code	Initial version: git tag v1-baseline and folder initial-version/. Final version: git tag v2-final (project root).

0. Starting point

I started from a small version of my Assignment #03 prototype. It has one room (Seminar Room B-204), three fictional bookings and a fixed current time: Monday, 2 November 2026, 09:30. These are the six acceptance criteria that I had to keep working:

AC	Rule
AC1	The time must be valid: HH:MM format, end after start, start not before 09:30, inside opening hours 08:00-20:00
AC2	A booking can be at most 120 minutes (120 is OK, 121 is not)
AC3	A blocked room is never shown as available
AC4	If a request only touches another booking it is OK, but a real overlap is rejected
AC5	Booking data cannot be changed by a check
AC6	The user gets a clear success or error message, and the page works on a phone

Before changing anything I ran the old tests: npm test on tag v1-baseline gave **10 of 10 passed** (saved in evidence/baseline-test-results.txt).

Task 1. Codebase analysis

First I wrote a short context file for the AI assistant (CONTEXT.md). It has a list of all files and what each one does, the acceptance criteria, the new requirements

F1-F5 and my constraints: no login, no API, no database, only fictional data, no build step, and small changes instead of rewriting everything.

Maintainability risks

All line numbers are from the old app.js.

#	**Problem**	**Where**	**Why it is bad**
R1	The same time parsing code is copied 9 times	checkAvailability() lines 22-24 and 43-44, renderBookings() lines 64-65, onCheckClick() lines 82-83	If I fix a bug in one copy, I can easily forget the other copies
R2	The UI checks the 120-minute rule by itself, before calling the real check	onCheckClick() line 84 and checkAvailability() line 35	I tested 07:00-09:30: the rule function says “in the past”, but the page says “at most 120 minutes”. Two places give two different answers
R3	Data, rules and UI are all in one file, with one hard-coded room	global ROOM and BOOKINGS (lines 9-15)	It is not possible to add more rooms without changing every function
R4	OPEN and CLOSE exist, but the code uses 8 * 60 and 20 * 60	lines 6-7 and line 32	Changing opening hours in the constant does nothing
R5	HTML is built from strings with innerHTML (5 times)	lines 68, 79, 85, 91, 93	The same message HTML is repeated, and booking titles are not escaped

My plan (written before editing, see docs/01-analysis-and-plan.md)

1. Move time parsing and settings into separate modules (R1, R4).
2. Keep only one validation function (R2).
3. Make a room catalog where every room has its own bookings (R3).
4. Add filter functions for capacity and equipment (F1, F2).
5. Split the UI into a render part and a controller (R3, R5, F4, F5).

6. Run the tests after every step and add new tests.

AI suggestions I did not accept as they were

- **Rejected:** the assistant suggested moving to ES modules and Vite. I checked it: when index.html is opened directly from disk, Chrome blocks module scripts because of CORS (evidence/esm-file-protocol-check.txt). Vite also needs a build step. So I used normal scripts that share one SmartCampus object and also export functions for Node tests.
- **Changed:** the assistant suggested removing blocked rooms inside the filter. Then the "blocked" rule would be in two places, which is the same mistake as R2. In my version the filter only checks capacity and equipment. Blocked rooms are still shown with a "Blocked" label, and only checkAvailability() refuses them.

Task 2. Feature extension

The new file src/data/rooms.js has 5 fictional rooms. Every room has an id, name, capacity, equipment list, blocked status and its own bookings. Room B-204 keeps the old bookings, so the old tests still make sense. Room E-115 is blocked and has no bookings, so only the blocked rule can stop it.

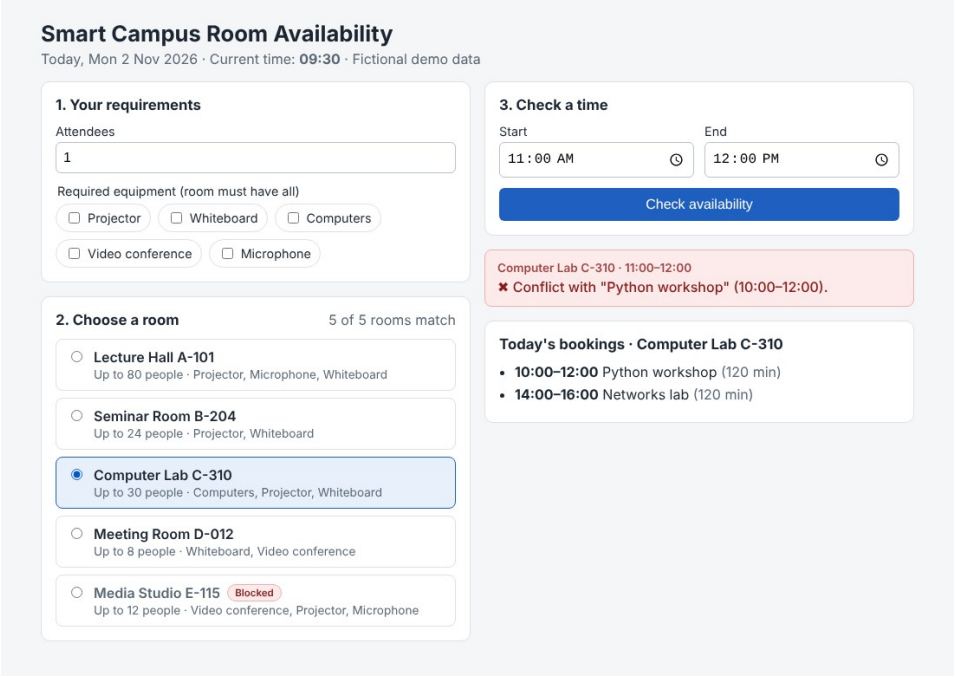
Room	Capacity	Equipment	Blocked
Lecture Hall A-101	80	projector, microphone, whiteboard	no
Seminar Room B-204	24	projector, whiteboard	no
Computer Lab C-310	30	computers, projector, whiteboard	no
Meeting Room D-012	8	whiteboard, video conference	no
Media Studio E-115	12	video conference, projector, microphone	yes

How each requirement works:

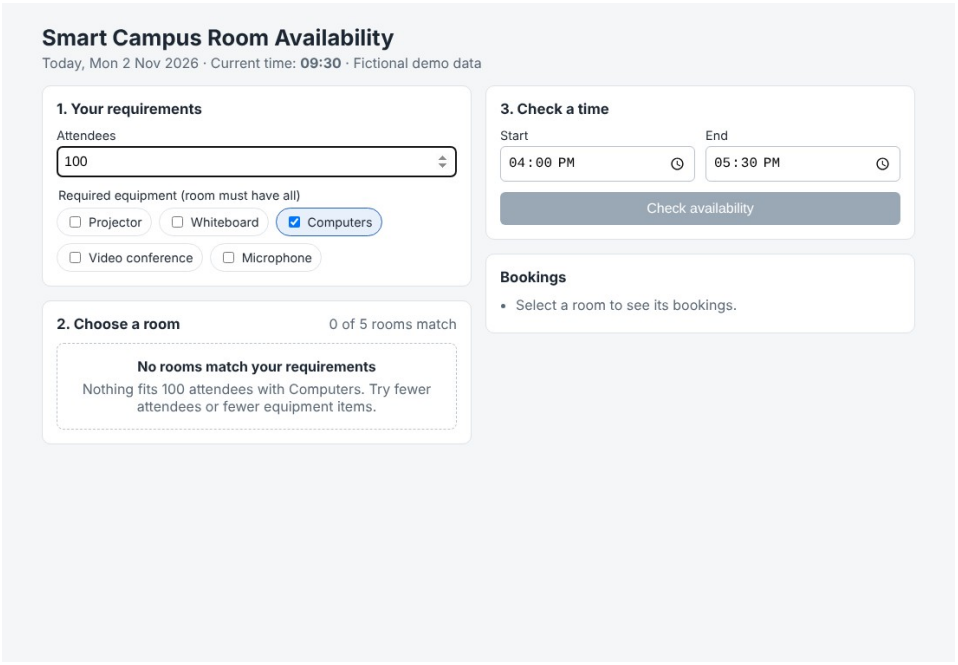
- **F1:** a room is shown only if its capacity is greater than or equal to the number of attendees.
- **F2:** the room must have all selected equipment, not just one item.
- **F3:** a blocked room can appear in the list, but the check always says it is blocked.

- **F4:** if no room matches, the page explains why and gives a hint. The check button is disabled.
- **F5:** the page remembers the last checked time. When the user picks another room, the check runs again for that room. If a filter hides the selected room, the first matching room is selected.

Screenshots of the demo are in evidence/screenshots/: a successful filter (02, 25 people and computers gives only C-310), an unsuccessful filter (03, 100 people gives the empty state), room switching for 11:00-12:00 (04 B-204 free, 05 C-310 conflict, 06 E-115 blocked), a failed check with 121 minutes (07) and the phone layout (08).



Switching to C-310: the same time now has a conflict with this room's own booking

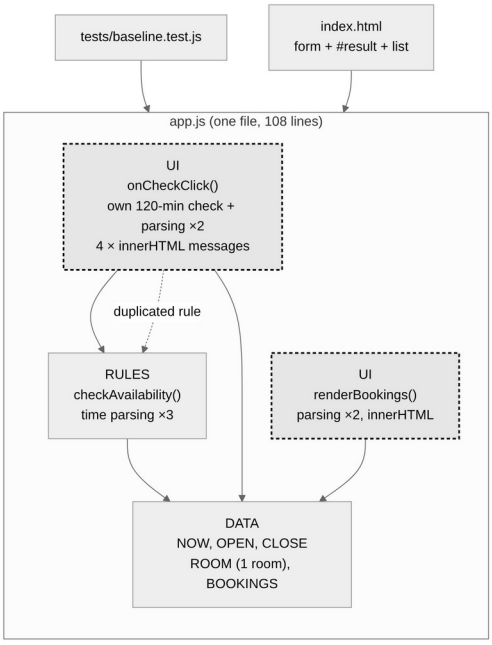


No room fits 100 people, so the page shows an empty state

Task 3. Refactoring

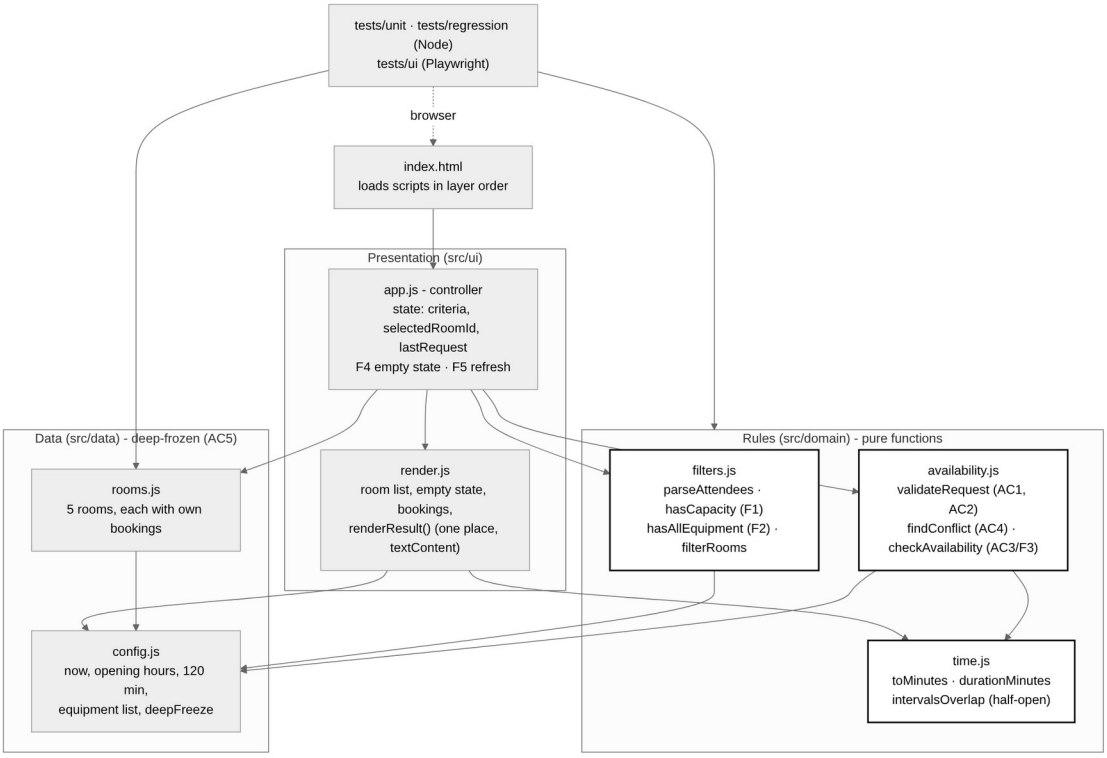
I did the refactoring in two commits and ran the tests after each one. In the first commit (“Refactor step 1”) I moved the settings, time functions and booking rules into their own files. I did not change the old tests at all, and they still passed 10 of 10 (evidence/step1-test-results.txt). The second commit added the catalog, the filters and the new UI.

Before: one file does everything.



Before

After: three layers (data, rules, UI). An arrow means “uses”.



After

Design decisions

1. **Every rule lives in one place.** The 120-minute rule, the blocked rule and time parsing each exist in only one function. The UI just shows the result. This fixed the R2 bug, and now the tests check exactly what the user sees.
2. **Frozen data and pure functions.** Rooms and bookings are frozen with deepFreeze(), so trying to change them throws an error. The rule functions only read data and return a new result. So AC5 is protected by the language itself, and I can test the rules in Node without a browser.
3. **Normal scripts instead of ES modules,** because the page must open with a double-click (see Task 1).
4. **The UI uses textContent and one renderResult() function.** All 5 innerHTML writes and the 4 copies of message HTML are gone.

Three things now work a little differently, on purpose, and they have tests: times like 24:00 or 12:60 are now “invalid format”; the “fill in both times” message moved from the UI into validateRequest(); and 07:00-09:30 now correctly says “in the past”.

Task 4. Automated testing

There are **34 tests**. The Node tests use the built-in test runner, and the UI tests use Playwright with a real Chromium browser.

Group	File	Tests	What they check
Regression	tests/regression/ac-regression.test.js	14	AC1-AC5: all 10 old cases, plus boundaries and the R2 case
Features	tests/unit/filters.test.js	11	room data, F1-F5, attendees input, filter does not change data
Time	tests/unit/time.test.js	2	parsing, duration, overlap
UI	tests/ui/ui.test.js	7	start page, filters, empty state, room switching, old rules in the page, phone width

Boundary cases: 120 and 121 minutes (AC2); touching on both sides, a normal overlap and an overlap of only one minute (AC4); 24 and 25 attendees for room B-204, which has exactly 24 seats (F1).

Command	Version	Result
npm test	initial (v1-baseline)	10 passed of 10
npm test with the old tests unchanged	after refactor step 1	10 passed of 10
npm run test:all	final	**34 passed of 34**, 0 failed
npm run mutation	final	**5 of 5 broken versions caught**

I also wanted to know if my tests can really fail. The mutation script breaks one rule at a time in a copy of the code (for example >= becomes > in the capacity check, or “all equipment” becomes “any equipment”). All 5 broken versions made the tests fail.

My first UI test run had one failure. The app was fine, but my test tried to type “abc” into a number field, and a browser does not allow that. I changed the test to use 2.5 and check for the “whole number” message.

Task 5. Quality evaluation

Measure	Initial	Final	How I measured it
Tests passed / total	10 / 10 (only AC1-AC5, F1-F5 did not exist yet)	34 / 34	npm test and npm run test:all, outputs in evidence/
Broken versions caught by tests	not measured	5 / 5	npm run mutation
Lines that parse HH:MM	9	1	scripts/metrics.sh (searches for split(":"))
Places that check the 120-minute rule	2	1	scripts/metrics.sh
Places that build the message HTML	4	1	scripts/metrics.sh
Duplicated logic removed		**12 copies** (8 + 1 + 3)	difference of the three rows above
innerHTML writes / magic numbers	5 / 1	0 / 0	scripts/metrics.sh
Structure	1 file, 96 lines, 4 jobs in one file	7 files in 3 layers, 435 lines, one job per file	scripts/metrics.sh, evidence/metrics.txt

The final version is longer, but that is because it has a whole new feature. I do not use the number of lines as a quality score.

What is still missing:

- There is only one day and a fixed clock. The app checks availability but cannot save a booking.
- The UI tests need Playwright and Chromium (npm install). Only the 27 Node tests run without anything extra.
- render.js is tested only through the browser tests, not on its own.
- On some computers <input type="time"> shows AM/PM, while the app uses 24-hour time.

Task 6. AI reflection

Before asking for any code, I gave the assistant a short CONTEXT.md file. It had a list of the files and what each one does, the six acceptance criteria written as clear rules, the new F1-F5 requirements, and my limits: no framework, no build step, fictional data only, and small steps instead of a full rewrite. I asked for an analysis and a plan first, and only then for code, one change at a time.

The most useful ideas were to have one time parser and one validation function, and to keep the rules as pure functions that tests can call directly. Thanks to this I found a real bug: for 07:00-09:30 the page and the rule function gave different messages. Two suggestions did not fit my project. Moving to ES modules with Vite would stop the page from opening from disk, and removing blocked rooms in the filter would put the blocked rule in two places again. I rejected the first one and changed the second one.

I checked the work myself. I ran all the test commands and read the output, compared the boundary cases with the booking data by hand, and opened the page on a laptop screen and at phone width. I also ran a mutation check to see if the tests can fail.

The tests affected my decisions. Because the old tests still passed without any changes after the first refactoring step, I knew the behaviour was the same. One failing UI test showed me that my test was wrong, not the app. The metrics script also needed a fix, because at first it counted one CSS class twice.

Next time I would add saving of bookings, unit tests for render.js, and a date picker instead of one fixed day.

Appendix A. AI-use record

Tool: Claude (Anthropic), model Claude Opus 5.5. Date: 31 October 2026. I did not share any passwords or private data.

#	What I asked	What I used	Problems	What I changed	How I checked
1	Do Assignment #04 (gave the PDF) and work from CONTEXT.md	Old one-room version and 10 AC tests	none	none	npm test 10/10, screenshot
2	Find at least 3	docs/01-	R2 needed	Tested 07:00-	The two

#	What I asked	What I used	Problems	What I changed	How I checked
	risks with line numbers and make a plan before editing	analysis-and-plan.md (R1-R5, 6 steps)	proof	09:30 in Node and in a browser	answers were different
3	Suggest a module structure	ES modules and Vite; remove blocked rooms in the filter	Does not work from disk; blocked rule in two places	Rejected the first, changed the second	esm-file-protocol-check.txt, F3 tests
4	Refactor step 1 only	config.js, time.js, availability.js	none	Kept the old function as a small wrapper	Old tests unchanged, 10/10
5	Add the catalog, filters and new UI	rooms.js, filters.js, render.js, ui/app.js, index.html	none	none	27 Node tests pass, checked screenshots
6	Write UI tests with Playwright	tests/ui/ ui.test.js	One test typed text into a number field	Used 2.5 instead	7/7 pass
7	Count duplicated code without guessing	scripts/ metrics.sh	It also counted msg-title	Fixed the search pattern and checked lines with grep -n	evidence/ metrics.txt
8	Check that the tests can fail	scripts/ mutation-check.sh	none	none	5/5 caught

Where to check everything: test outputs in evidence/*.txt, screenshots in evidence/screenshots/, git log (one commit per step), and evidence/esm-file-protocol-check.txt.

Appendix B. Optional: using an existing Agent Skill

I used the existing **bug-detector** skill from Practice #08 to investigate problem R2. I did not create a new SKILL.md. This is the result in the skill's format:

Defect	**Evidence**	**Root cause**	**Fix**	**Test**
For 07:00-09:30 the page says "at most 120 minutes", but the start time is actually in the past	Node: checkAvailability(...) returns IN_PAST; the browser shows "A booking can be at most 120 minutes."	onCheckClick() (line 84) checks the 120-minute rule before calling the real check	Remove the check from the UI, the UI only shows the result	"UI and domain use one validation order" and the UI test case 07:00-09:30